

- Module 1
AI Agents, Tools, Actions, & Language
- Module 2
GPT-4: A Conceptual Framework for AI Agents
- Module 3
Agent Tool Management
- Module 4
Orchestrating New Software as Built in the Age of AI Agents

Possible Solution for Building a Quasi-Agent

Understanding the Architecture

Our quasi-agent works through three key steps:

1. Initial code generation
2. Documentation enhancement
3. Test case creation

The magic happens in how we maintain context between these steps, ensuring each builds on the previous results.

1. Initial code execution.

The magic happens in how we maintain context between these steps, ensuring each build is on the previous results.

Let's break down the key pieces:

```

        return response.choices[0].message.content
    except Exception as e:
        print(f"Error: {e}")
        return None

```

```
def extract_code_block(response: str) -> str:
    """Extract code block from response"""
    if not "```" in response:
        return response

    code_block = response.split("```")[1].strip()

    if code_block.startswith("python"):
        code_block = code_block[6:]

    return code_block
```

The main function, `develop_vector_functions()`, orchestrates these phases of development:

```
messages = [
    ("role": "system", "content": "You are a Python expert helping to develop a function.")
]

messages.append([
    ("role": "user",
     "content": f"Write a Python function that {function_description}. Output the function in a python code block."
    )
])
```

```
Phase 2: Documentation Enhancement

myutils.append([
    "title": "initutils",
    "content": "\n\n\ndef __init__(self):
    self.initial_function = self.__init__\n\n"
])

myutils.append([
    "title": "user",
    "content": "\n\n\ndef __init__(self):
    self.add_comprehensive_documentation_to_this_function...\n\n"
])
```

```
Phase 3: Test Case Generation
```

```
messages.append([  
    "id": "associated",  
    "content": "\t\t\t\tython(x) documented function \"x\" \t\t\t"]]  
  
messages.append([  
    "id": "user",  
    "content": "Add unit test cases for this function..."]]  

```

Memory Management Through Message History

The key insight is how we manage "memory" through the messages list. Each step builds on previous responses, but we carefully control what the LLM sees:

1. We only show the code, not the commentary
2. Each message provides specific instruction for the next enhancement
3. The context builds progressively through the message history

For example, when adding documentation, the LLM sees:

- It's a Python script (system message)
- The original code (previous response)
- The request for documentation (current task)

This focused context helps ensure consistent, high-quality output.

```

''' Here's how it works in practice '''

def fibonacci_sequence(n, fibname, do_print=False):
    """
    Return the n-th Fibonacci number.
    Example: "5" returns the 5th Fibonacci number, 5.
    Your observation: Calculate Fibonacci sequence up to n
    """
    # Initial Fibonacci
    fib = 0
    fibname[0] = 0
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        sequence = [0, 1]
        while sequence[-1] < n:
            sequence.append(sequence[-1] + sequence[-2])
        return sequence

# Decoupled Function
''' Function with added documentation '''

# Test Cases
'''
- calculate first cases
'''

Final code has been saved to calculate_fibonacci_sequence.py

```

- 1. **Prompt Chaining:** Breaking complex tasks into sequential steps makes them more manageable.
- 2. **Content Management:** Carefully controlling what the LLM sees helps maintain focus and consistency.
- 3. **Output Processing:** Having robust ways to extract and clean LLM output is crucial.
- 4. **Progressive Enhancement:** Building features iteratively (code → docs → tests) creates better results than trying to do everything at once.

These principles apply even when building more complex, fully agentic systems.

[illegible]